Chapitre 4 — Les jointures : relier plusieurs tables

Table of Contents

1. Situation professionnelle . . . 1
2. Objectifs du chapitre . . . 1
3. Rappels : clés et modèle relationnel . . . 2
4. Partie 1 — Le produit cartésien (à éviter) . . . 2
5. Partie 2 — La jointure interne : `INNER JOIN` . . . 3
6. Partie 3 — La jointure externe : `LEFT JOIN` . . . 4
7. Partie 4 — La jointure externe avec restriction . . . 4
8. Partie 5 — Jointures multiples . . . 5
9. Structure d'une requête avec jointures . . . 6
10. Synthèse . . . 7
11. Exercices . . . 7
    11.1. Exercice 1 [*] . . . 7
    11.2. Exercice 2 [**] . . . 7
    11.3. Exercice 3 [*] . . . 7
12. Corrigés . . . 7

Les données sont réparties dans **plusieurs tables** pour éviter la redondance. Une **jointure** relie ces tables via une **clé primaire** d'un côté et une **clé étrangère** de l'autre. On étudie ici la jointure interne (`INNER JOIN`), la jointure externe (`LEFT JOIN`) et la jointure externe **avec restriction**. Jeu de données : **Football**.

1. Situation professionnelle

L'application de la ligue stocke les pays, les continents, les équipes, les joueurs et les matchs dans des tables séparées. Pour afficher « le pays de chaque équipe », « les joueurs du FC Barcelone avec leur nationalité » ou « les pays qui n'ont aucune équipe », il faut **combiner** ces tables : c'est le rôle des jointures.

2. Objectifs du chapitre

- comprendre clé primaire / clé étrangère et le **modèle relationnel** ;
- comprendre le **produit cartésien** et pourquoi un critère de jointure est indispensable ;
- écrire une **jointure interne** (`INNER JOIN ⋯ ON ⋯`) ;

- écrire une **jointure externe** (`LEFT JOIN`) et sa variante **avec restriction** (`… WHERE clé IS NULL`) ;
- enchaîner **plusieurs jointures**.

3. Rappels : clés et modèle relationnel

Clé primaire

Colonne (ou champ) qui **identifie de façon unique** chaque ligne d'une table. Pas de doublon, une seule par table.

Clé étrangère

Colonne qui **référence la clé primaire d'une autre table**. Elle matérialise une association. Une table peut avoir plusieurs clés étrangères.

Le **modèle relationnel** du jeu **Football** (extrait) :

```
continent (id_continent, nom_continent)
  Clé primaire : id_continent

pays (id_pays, nom_pays, fk_id_continent)
  Clé primaire : id_pays
  Clé étrangère : fk_id_continent → continent(id_continent)

equipes (id_equipe, fk_id_pays, nom_equipe, nom_stade, capacite_stade)
  Clé primaire : id_equipe
  Clé étrangère : fk_id_pays → pays(id_pays)

joueurs (id_joueur, fk_id_pays, fk_id_equipe, nom_joueur, prenom_joueur,
date_naissance, salaire)
  Clé primaire : id_joueur
  Clés étrangères : fk_id_equipe → equipes(id_equipe) ; fk_id_pays → pays(id_pays)

matches (id_match, fk_id_equipe_local, fk_id_equipe_visite, date_match,
         score_equipe_local, score_equipe_visite)
  Clé primaire : id_match
  Clés étrangères : fk_id_equipe_local → equipes(id_equipe)
                    fk_id_equipe_visite → equipes(id_equipe)
```

Script complet : `scripts/football.sql`. Positionnement : `SET search_path TO football;`.

4. Partie 1 — Le produit cartésien (à éviter)

Si l'on interroge deux tables **sans critère de jointure**, le moteur combine **chaque** ligne de la première avec **chaque** ligne de la seconde : c'est le **produit cartésien**. Une table de m lignes et une de n lignes donnent $m \times n$ lignes — presque toujours une erreur.

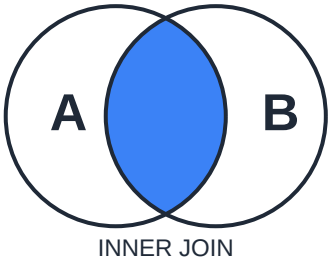
```
-- 19 équipes x 23 pays = 437 lignes, la plupart n'ont aucun sens
```

```
SELECT e.nom_equipe, p.nom_pays
FROM equipes e
CROSS JOIN pays p;
```

Le critère de jointure (clause `ON`) sert justement à **ne garder que les combinaisons pertinentes**.

5. Partie 2 — La jointure interne : `INNER JOIN`

La jointure interne ne conserve que les lignes où le critère `ON` est **vérifié des deux côtés** : c'est l'**intersection**.




```
SELECT e.nom_equipe, p.nom_pays
FROM equipes e
INNER JOIN pays p ON p.id_pays = e.fk_id_pays
ORDER BY e.nom_equipe
LIMIT 6;
```

Table 1. Résultat (6 premières lignes)

nom_equipe	nom_pays
AS Roma	Italie
Al Hilal	Arabie Saoudite
Al-Arabi SC	Qatar
Al-Ittihad	Arabie Saoudite
Atalanta	Italie
Atlético Madrid	Espagne

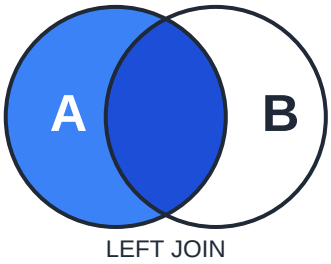


Dans un critère de jointure, on trouve **toujours** d'un côté une **clé primaire** (`p.id_pays`) et de l'autre une **clé étrangère** (`e.fk_id_pays`). Les deux colonnes portent souvent des noms différents : seul le **modèle relationnel** permet de les retrouver.

On **préfixe chaque colonne par l'alias de sa table** (`e.nom_equipe`, `p.nom_pays`), ce qui lève toute ambiguïté quand deux tables ont des colonnes de même nom.

6. Partie 3 — La jointure externe : `LEFT JOIN`

La jointure externe gauche conserve **toutes les lignes de la table de gauche**, même celles sans correspondance à droite. Les colonnes de droite valent alors `NULL`.




```
SELECT p.nom_pays, e.nom_equipe
FROM pays p
LEFT JOIN equipes e ON e.fk_id_pays = p.id_pays
WHERE p.id_pays IN (2, 3, 4, 5)
ORDER BY p.nom_pays, e.nom_equipe;
```

Table 2. Résultat

nom_pays	nom_equipe
Argentine	*(NULL)*
Brésil	*(NULL)*
Espagne	Atlético Madrid
Espagne	FC Barcelone
Espagne	Real Madrid
France	Olympique de Marseille
France	Paris Saint Germain

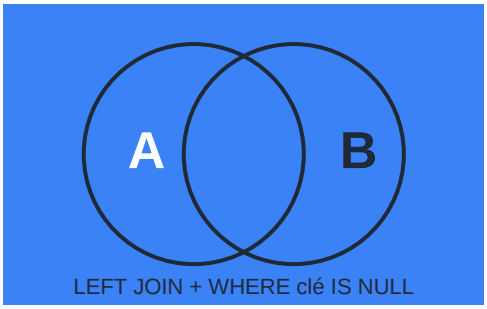
L'Argentine et le Brésil apparaissent **sans équipe** (aucune équipe du jeu n'y est rattachée) : la colonne `nom_equipe` est `NULL`. Avec un `INNER JOIN`, ces deux pays **disparaîtraient**.



`LEFT JOIN` (= `LEFT OUTER JOIN`). Il existe aussi `RIGHT JOIN` (tout garder à droite) : `A RIGHT JOIN B` équivaut à `B LEFT JOIN A`. On privilégie `LEFT JOIN`, plus lisible.

7. Partie 4 — La jointure externe avec restriction

C'est le cas le plus **fréquent en examen** : trouver les lignes de gauche **sans correspondance** à droite (les « pays sans équipe », « films jamais vendus »…). On fait un `LEFT JOIN` puis on **garde uniquement les** `NULL` avec `WHERE clé_étrangère IS NULL`.




```
SELECT p.nom_pays
FROM pays p
LEFT JOIN equipes e ON e.fk_id_pays = p.id_pays
WHERE e.id_equipe IS NULL
ORDER BY p.nom_pays;
```

Table 3. Résultat (15 pays sans équipe)

nom_pays
Argentine
Autriche
Belgique
Brésil
Canada
Croatie
Egypte
Géorgie
Nigeria
Norvège
Pays-Bas
Pologne
Portugal
Slovénie
Écosse



On teste `IS NULL` sur une colonne **toujours renseignée** côté droit quand il y a correspondance — idéalement la **clé primaire** de la table de droite (`e.id_equipe`). Tester une colonne éventuellement nulle par nature fausserait le résultat.

8. Partie 5 — Jointures multiples

Pour traverser plusieurs associations, on **enchaîne les `JOIN`**. Exemple : les joueurs d'une équipe **avec leur nationalité** (deux clés étrangères de `joueurs` : vers `equipes` et vers `pays`).

```
SELECT j.nom_joueur, e.nom_equipe, pn.nom_pays AS nationalite
FROM joueurs j
INNER JOIN equipes e ON e.id_equipe = j.fk_id_equipe
INNER JOIN pays pn   ON pn.id_pays  = j.fk_id_pays
WHERE e.id_equipe = 2            -- FC Barcelone
ORDER BY j.nom_joueur;
```

Table 4. Résultat

nom_joueur	nom_equipe	nationalite
DE JONG	FC Barcelone	Pays-Bas
GAVI	FC Barcelone	Espagne
LEWANDOWSKI	FC Barcelone	Pologne
PEDRI	FC Barcelone	Espagne
TORRES	FC Barcelone	Espagne

On peut aussi passer par une **table intermédiaire** uniquement pour atteindre une restriction, même si aucune de ses colonnes n'est affichée. Exemple : les joueurs dont la **nationalité est sud-américaine** (on traverse `pays` puis `continent`) :

```
SELECT j.nom_joueur, p.nom_pays, c.nom_continent
FROM joueurs j
INNER JOIN pays p      ON p.id_pays = j.fk_id_pays
INNER JOIN continent c ON c.id_continent = p.fk_id_continent
WHERE c.nom_continent = 'Amérique du Sud'
ORDER BY j.nom_joueur;
```

Table 5. Résultat

nom_joueur	nom_pays	nom_continent
ALISSON	Brésil	Amérique du Sud
MARQUINHOS	Brésil	Amérique du Sud
MARTINEZ	Argentine	Amérique du Sud
NEYMAR	Brésil	Amérique du Sud
PAREDES	Argentine	Amérique du Sud
VINICIUS	Brésil	Amérique du Sud

9. Structure d'une requête avec jointures

Clause	Rôle
`SELECT`	colonnes du résultat (préfixées par l'alias)

Clause	Rôle
`FROM`	table principale
`INNER/LEFT JOIN`	table supplémentaire à joindre (0 à n)
`ON`	critère de jointure (clé primaire = clé étrangère)
`WHERE`	filtre sur les lignes (dont `… IS NULL` pour les restrictions)
`ORDER BY`	tri du résultat

10. Synthèse

- Jointure = clé primaire d'un côté, clé étrangère de l'autre, dans la clause `ON`.
- `INNER JOIN` : intersection (uniquement les correspondances).
- `LEFT JOIN` : tout à gauche + correspondances à droite (sinon `NULL`).
- `LEFT JOIN … WHERE clé_droite IS NULL` : les lignes de gauche **sans** correspondance.
- Jointures multiples : un `JOIN … ON …` par table traversée, même en simple passage.
- Toujours **préfixer** les colonnes par l'alias de table.

11. Exercices

```
SET search_path TO football;
```

11.1. Exercice 1 [*]

Afficher le nom de chaque joueur (`nom_joueur`) avec le nom de son équipe (`nom_equipe`), uniquement pour les joueurs de **Manchester City** (`id_equipe = 7`). Trier par nom de joueur.

11.2. Exercice 2 [**]

Afficher pour chaque **match** le nom de l'équipe locale et de l'équipe visiteuse, ainsi que le score, pour les matchs du **20 septembre 2025 au 23 septembre 2025** inclus. Trier par date.

11.3. Exercice 3 [*]

Afficher les équipes qui **n'ont disputé aucun match à domicile** (aucune ligne dans `matches` avec `fk_id_equipe_local` = leur identifiant). Afficher `nom_equipe`, trié par nom.

12. Corrigés

▼ *Details*

Corrigé exercice 1

```
SELECT j.nom_joueur, e.nom_equipe
```

```
FROM joueurs j
INNER JOIN equipes e ON e.id_equipe = j.fk_id_equipe
WHERE e.id_equipe = 7
ORDER BY j.nom_joueur;
```

Table 6. Résultat

nom_joueur	nom_equipe
BERNARDO	Manchester City
FODEN	Manchester City
GREALISH	Manchester City
HAALAND	Manchester City
RODRI	Manchester City
WALKER	Manchester City

Corrigé exercice 2

```
SELECT el.nom_equipe AS local, ev.nom_equipe AS visiteur,
       m.date_match, m.score_equipe_local, m.score_equipe_visite
FROM matches m
INNER JOIN equipes el ON el.id_equipe = m.fk_id_equipe_local
INNER JOIN equipes ev ON ev.id_equipe = m.fk_id_equipe_visite
WHERE m.date_match BETWEEN DATE '2025-09-20' AND DATE '2025-09-23'
ORDER BY m.date_match;
```

Table 7. Résultat

local	visiteur	date_match	score_equipe_local	score_equipe_visite
Bayern Munich	Paris Saint Germain	2025-09-20	2	1
Manchester City	Atalanta	2025-09-21	4	0
Al-Ittihad	Chelsea	2025-09-22	2	2
Bayern Munich	Real Madrid	2025-09-23	3	0



La table `equipes` est jointe **deux fois** (alias `el` et `ev`) : une fois pour l'équipe locale, une fois pour la visiteuse. C'est indispensable dès qu'une table est référencée par deux clés étrangères.

Corrigé exercice 3

```
SELECT e.nom_equipe
FROM equipes e
LEFT JOIN matches m ON m.fk_id_equipe_local = e.id_equipe
WHERE m.id_match IS NULL
```

```
ORDER BY e.nom_equipe;
```

Table 8. Résultat

nom_equipe
Atalanta
Chelsea